

JD Developer CRM

Centro de control para agencia de desarrollo web, IA y automatización

Arquitectura · Stack · Conexiones · Flujos · Seguridad

Versión 1.0.0 · Generado el 17 de julio de 2026

Índice

1. Qué es el proyecto
2. Lenguaje y stack tecnológico
3. Arquitectura de conexiones
4. Google Sheets como base de datos
5. Los workflows de n8n (el "backend")
6. Cómo fluye una acción (paso a paso)
7. Organización del código
8. Módulos funcionales del CRM
9. Gestión de estado y datos
10. Autenticación
11. Desarrollo vs. Producción
12. Integraciones externas
13. Seguridad y hoja de ruta
14. Resumen ejecutivo

1. Qué es el proyecto

El **JD Developer CRM** es el centro de control de la agencia. En una sola aplicación web permite:

- **Captar leads** automáticamente (scraping de Google Maps, web, redes) y desde el formulario de la web pública.
- **Gestionar el pipeline de ventas** con un tablero Kanban de arrastrar y soltar.
- **Enviar outreach y responder correos** (SMTP/IMAP) con redacción asistida por IA.
- **Organizar campañas, tareas y seguimientos.**
- **Analizar métricas** (embudo de conversión, forecast por probabilidad).
- **Puntuar oportunidades con IA** (Claude asigna un score 0–100).
- Todo con **borrado suave** y papelera de recuperación.

Idea clave: no tiene un backend propio programado a mano. El "servidor" es **n8n**, que hace de proxy seguro contra **Google Sheets**, la base de datos real del sistema.

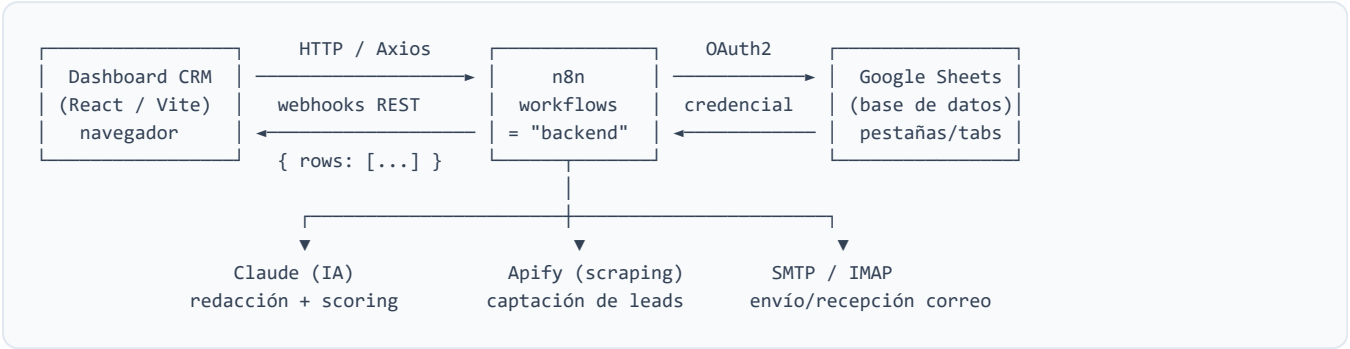
2. Lenguaje y stack tecnológico

Es una **SPA (Single Page Application) 100% frontend** escrita en **TypeScript + React 18**, empaquetada con Vite.

Capa	Tecnología	Para qué
Lenguaje	TypeScript 5.7	Tipado estático en todo el código
UI	React 18 + JSX	Componentes de interfaz
Build / dev	Vite 6	Servidor de desarrollo y empaquetado
Estilos	Tailwind CSS 3 + PostCSS	Diseño con utilidades CSS
Enrutado	React Router DOM 6	Navegación entre páginas
Estado servidor	TanStack React Query 5	Caché, refetch automático (30s)
Estado local	Zustand 5	Estado global + persistencia localStorage
HTTP	Axios	Llamadas a los webhooks de n8n
Formularios	React Hook Form + Zod	Formularios y validación tipada
Gráficos	Recharts	Analítica y embudos
Drag & drop	@dnd-kit	Kanban del pipeline
Animación	Framer Motion	Transiciones de UI
Notificaciones	react-hot-toast	Avisos tipo toast
Internacionalización	i18next / react-i18next	Multiidioma (es / en)
Iconos	lucide-react	Iconografía

3. Arquitectura de conexiones

El corazón del diseño. El dashboard **nunca** habla directamente con Google Sheets; siempre pasa por webhooks REST de n8n:



Endpoints principales

- **Lectura:** `GET /webhook/crm-sheets-read?sheet=:tab`
- **Escritura:** `POST /webhook/crm-sheets-write` con cuerpo `{ action, leadId, ... }`

Cada operación de negocio (crear lead, mover etapa, puntuar con IA, enviar correo) es un **action** distinto que llega al mismo webhook de escritura.

¿Por qué así? Para **no incrustar una Google API key en el bundle del frontend**. n8n guarda la credencial OAuth2 de Google de forma segura y el navegador solo llama a webhooks. Esto separa las credenciales sensibles del código que se descarga al cliente.

4. Google Sheets como base de datos

Cada "tabla" es una **pestaña (tab)** de un Google Sheet. Las pestañas usadas:

Pestaña	Contenido
prospects	Leads captados (empresa, contacto, métricas web, score)
pipeline	Estado de cada lead en el embudo de ventas
outreach	Correos de captación (asunto, cuerpo, estado de envío)
messages	Historial de mensajes / actividad
inbox	Correos recibidos vía IMAP
campaigns	Campañas de outreach
web_leads	Solicitudes del formulario de la web pública
tareas	Tareas y seguimientos manuales
contactos	Contactos por lead (varios por empresa)
notas	Historial de notas por lead
config	Configuración clave/valor (p. ej. alias de correo)
search_log	Historial de búsquedas de leads disparadas

Borrado suave (soft-delete): nada se borra físicamente. Se marcan columnas `Eliminado` / `Purgado`. Por eso existe la **Papelera**, que permite restaurar o purgar definitivamente (purga lógica, sin borrar la fila).

5. Los workflows de n8n (el "backend")

Los flujos viven como JSON en la carpeta `n8n/` y se importan a n8n. Cada uno expone webhooks:

Workflow	Función
<code>CRM_API_Leer_Sheets</code>	Lee cualquier pestaña como JSON <code>{ rows: [...] }</code>
<code>CRM_API_Escribir_Sheets</code>	CRUD por <code>action</code> : crear/editar/eliminar leads, pipeline, campañas, contactos, notas, config
<code>CRM_API_Web_Lead</code>	Gestiona solicitudes del formulario web
<code>CRM_API_Tareas</code>	CRUD de tareas y seguimientos
Endpoints IA	<code>/crm-generate-ai</code> , <code>action: analizar_lead</code> , <code>puntuar_lead</code> → llaman a Claude
Correo	<code>/crm-send-reply</code> → envío SMTP con adjuntos (base64)
Captación	<code>/crm-buscar-leads</code> → dispara scraping con Apify

El cliente TypeScript que consume todo esto es `src/services/crmApi.ts`: un objeto con un método por cada acción, todos apoyados en una instancia de Axios con `baseUrl` y un header opcional `X-CRM-TOKEN`.

6. Cómo fluye una acción (paso a paso)

Ejemplo real: **convertir una solicitud web en lead** (hook `useConvertWebLead`):

1. Un componente React invoca el hook (una *mutation* de React Query).
2. El hook llama a `crmApi.createLead()` + `updatePipeline()` + `updateWebLead()` (POST vía Axios a los webhooks).
3. n8n recibe cada petición y escribe en Google Sheets usando su credencial OAuth2.
4. Al terminar, React Query **invalida** las queries `['leads']` y `['web_leads']`.
5. La UI se re-hidrata automáticamente con los datos nuevos.

```
// Escritura optimista + persistencia
await crmApi.createLead({ leadId, empresa, emailContacto, ... })
await crmApi.updatePipeline({ leadId, estado: 'contactado', ... })
await crmApi.updateWebLead({ id, estado: 'cerrado', etiquetas })
// onSuccess → queryClient.invalidateQueries(['leads'])
```

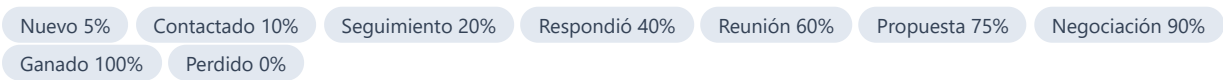
7. Organización del código (`src/`)

Carpeta	Rol
<code>features/</code>	Un módulo por carpeta: dashboard, leads, pipeline, campaigns, inbox, webleads, tareas, messages, automations, analytics, settings, trash, auth. Cargadas con <i>lazy loading</i> .
<code>services/</code>	Capa de red: <code>crmApi.ts</code> (webhooks), <code>n8nService.ts</code> (API de n8n), <code>sheetsService.ts</code> (normaliza filas → objetos tipados)
<code>hooks/useData.ts</code>	Todos los hooks de datos con React Query centralizados
<code>store/</code>	Estado global Zustand: authStore, leadsStore, campaignsStore, uiStore
<code>lib/config.ts</code>	Config central desde variables de entorno <code>VITE_*</code> + etapas del pipeline
<code>components/</code>	Layout (Sidebar, Topbar, CommandPalette) y UI reutilizable
<code>types/index.ts</code>	Todos los tipos TypeScript del dominio

8. Módulos funcionales del CRM

Módulo	Qué hace
Dashboard	KPIs y actividad reciente
Leads	Tabla de prospectos, drawer de detalle, favoritos, score IA manual y filtros
Pipeline	Kanban drag & drop por etapas, con forecast por probabilidad
Campañas	Asistente de creación (wizard) con plantillas de arranque
Inbox	Correos entrantes (IMAP), conversión a lead en un clic
Web Leads	Solicitudes del formulario público con estado/prioridad/etiquetas
Tareas	Seguimientos con vencimiento, prioridad y responsable
Mensajes	Redacción y envío de correos (SMTP)
Automatizaciones	Vista de workflows y ejecuciones de n8n
Analítica	Embudo de conversión y métricas (Recharts)
Configuración	Ajustes clave/valor, alias de correo
Papelera	Restaurar o purgar registros con soft-delete

Etapas del pipeline (con probabilidad para forecast)



9. Gestión de estado y datos

Dos capas complementarias:

- **React Query** — estado del *servidor*: cachea las respuestas de Sheets/n8n, refresca cada 30 segundos (`refetchInterval`), y tras cada escritura invalida las queries afectadas para mantener la UI sincronizada.
- **Zustand** — estado *local/global*: sesión de usuario, leads hidratados, campañas, estado de UI. La sesión se persiste en `localStorage` .

Las escrituras son **optimistas** en el store local: la UI reacciona al instante y, cuando el webhook responde, quedan además persistidas en Sheets.

10. Autenticación

Versión actual (v1): **contraseña única** validada en el cliente y persistida en `localStorage` . La estructura ya contempla multiusuario con roles (`admin` / `vendedor` / `viewer`) y 2FA a futuro.

Punto crítico de seguridad: al validarse en el frontend, el login es falsificable. La Fase 2 prevé mover la protección a **Cloudflare Access** para el equipo interno (gratuito), dejando Supabase para un futuro portal de clientes.

11. Desarrollo vs. Producción

En desarrollo

Vite proxya las rutas `/n8n-api` y `/n8n-hook` hacia n8n para **evitar CORS** y no exponer la API key al navegador:

```
server.proxy = {
  '/n8n-api':  → n8n /api/v1    // API pública de n8n
  '/n8n-hook': → n8n /webhook  // webhooks del CRM API
}
```

En producción

- Se compila estático (`tsc -b && vite build`) con `base: './'` para vivir en `jddeveloper.com/crm` .
- Se sirve detrás de **nginx** (ver `deploy/`), que inyecta `X-N8N-API-KEY` del lado servidor — así la key sale del bundle.
- **CI/CD** con GitHub Actions (`.github/workflows/`) y contenedor Docker (`deploy/Dockerfile`).

12. Integraciones externas

Servicio	Uso en el CRM
Google Sheets	Base de datos (vía OAuth2 en n8n)
n8n	Backend / orquestador de workflows y webhooks
Claude (IA)	Redacción de outreach, análisis y scoring de leads (0–100)
Apify	Scraping de prospectos (Google Maps, web, redes)
SMTP / IMAP (Hostinger)	Envío y recepción de correo
Cloudflare (Fase 2)	Access / protección del backoffice

13. Seguridad y hoja de ruta

Puntos identificados en la auditoría técnica:

- **Crítico** Login falsificable (validación en cliente).
- **Crítico** API key de n8n en el bundle → se resuelve con proxy nginx server-side.
- **Crítico** Webhooks sin autenticación → mitigable con **X-CRM-TOKEN**.

Fase 2 (planeada): Cloudflare Access para el equipo interno; migración futura de Sheets → Supabase solo cuando se solicite explícitamente; portal de clientes con Supabase Auth.

14. Resumen ejecutivo

Es una **SPA React + TypeScript** (Vite / Tailwind) que usa **React Query + Zustand** para el estado y que, en lugar de un backend propio, se comunica por **webhooks de n8n**. n8n hace de proxy seguro contra **Google Sheets** (la base de datos) e integra **Claude (IA)**, **Apify (scraping)** y **SMTP/IMAP (correo)** — todo para automatizar la captación y la gestión de clientes de la agencia.